Figure 4.1 Example of a middleware pipeline. Each middleware component handles the request and passes it on to the next middleware component in the pipeline. After a middleware component generates a response, it passes the response back through the pipeline. When it reaches the ASP.NET Core web server, the response is sent to the user's browser.

DEFINITION This arrangement—whereby a piece of middleware can call another piece of middleware, which in turn can call another, and so on—is referred to as a *pipeline*. You can think of each piece of middleware as being like a section of pipe; when you connect all the sections, a request flows through one piece and into the next.

One of the most common use cases for middleware is for the cross-cutting concerns of your application. These aspects of your application need to occur for every request, regardless of the specific path in the request or the resource requested, including

- Logging each request
- Adding standard security headers to the response
- Associating a request with the relevant user
- Setting the language for the current request

In each of these examples, the middleware receives a request, modifies it, and then passes the request on to the next piece of middleware in the pipeline. Subsequent middleware could use the details added by the earlier middleware to handle the request in some way. In figure 4.2, for example, the authentication middleware associates the request with a user. Then the authorization middleware uses this detail to verify whether the user has permission to make that specific request to the application.